Build a RAG-Powered Customer Support Agent

A 3-hour workshop guide

Instructor: Eng. Youssef Bastawisy Stack: Python · LangChain · LangGraph · Chroma · Streamlit · Groq

---

Overview

In three hours, every participant will build and deploy a production-shaped customer-support assistant for a fictional cloud-storage company called **CloudNest**. The agent retrieves answers from a custom knowledge base, decides when to use tools, remembers conversation context across turns, and ships to a public URL on Streamlit Community Cloud.

The codebase is intentionally three short files — one per architectural layer:

```
rag.py    -> data layer:    load -> chunk -> embed -> store -> retrieve
agent.py  -> reasoning:     LLM + tools + system prompt -> ReAct loop
app.py    -> UI:            Streamlit chat surface, session memory, streaming
```

Three files, three jobs. This separation is the single most undertaught lesson in LLM tutorials.

Pre-workshop email (send 24h ahead)

Send this the night before so the workshop doesn't lose 30 minutes to environment setup.

> **Before the workshop, please:**
>
> 1. Install **Python 3.11+**.
> 2. Sign up for a free Groq account at https://groq.com and create an API key.
> 3. Clone the starter repo and run:
>
>  ```
>  python -m venv .venv
>  .venv\Scripts\Activate.ps1     # PowerShell on Windows
>  # or:  source .venv/bin/activate   on macOS/Linux
>  pip install -r requirements.txt
>  python rag.py
>  ```
>
> The last command will download a ~90 MB embedding model and verify everything works. **Do this on your home Wi-Fi**, not at the venue.

Master agenda

Block	Time	Topic	Outcome
1	0:00 – 0:15	Welcome, demo, setup check	Everyone sees the working product
2	0:15 – 0:30	Concept primer: RAG and agents	Shared mental model
3	0:30 – 1:05	`rag.py` — the retrieval layer	Students run `python rag.py` and get hits
	1:05 – 1:15	**Break (10 min)**	Stretch + reset state
4	1:15 – 2:00	`agent.py` — the reasoning layer	Students run `python agent.py` end-to-end
5	2:00 – 2:35	`app.py` — the UI layer	Students run `streamlit run app.py`
6	2:35 – 2:50	Deploy to Streamlit Community Cloud	Each student has a public URL
7	2:50 – 3:00	Q&A and extensions	Students leave with three ideas to keep going

Block 1 — Welcome & demo (0:00 – 0:15)

Goal: hook the audience with the working product before any code.

Script (≈5 min)

> "Today we are building a real-feeling customer support assistant for a fictional cloud storage company called CloudNest. By the end of three hours, your laptop will be running this:" — *open the deployed Streamlit app you prepared, ask "tell me about pricing", show the answer with citation and the 'Tools used' expander*.
>
> "Notice three things. First, it is grounded in a knowledge base — it cites which file the answer came from. Second, it uses tools — pricing came from `search_knowledge_base`. Third, it remembers — ask 'and the cheapest one?' as a follow-up."

Setup verification (≈10 min)

Have students run, in their cloned project:

```
python -c "import langchain, langgraph, streamlit, chromadb; print('ok')"
```

If anyone gets `ModuleNotFoundError`, pair them with a neighbor who is already set up. **Do not debug installs from the front of the room** — it loses everyone else.

Block 2 — Concept primer (0:15 – 0:30)

Goal: every student leaves this block able to draw the architecture on a napkin.

Slide 1 — Why RAG?

> "LLMs hallucinate. They do not know your private data. They go stale. RAG fixes all three by **retrieving relevant text from your data first, then asking the LLM to answer using only that text**."

Slide 2 — The RAG pipeline (whiteboard)

```
[Markdown files] -> chunker -> [chunks] -> embeddings -> [vector store]
                                                              |
                                                              v
[user query] -> embedding -> similarity search -> [top-k chunks] -> LLM prompt -> answer
```

Slide 3 — Why an *agent* on top of RAG?

> "Plain RAG always retrieves, even for 'hi' or 'thanks'. An agent decides **whether** to retrieve, **when** to retrieve, and **what other tools** to use — like creating a support ticket. Today we use LangGraph's `create_react_agent`, which is the simplest agent abstraction that exists."

Block 3 — `rag.py` deep dive (0:30 – 1:05)

Goal: students understand every line of `rag.py` and can run it standalone.

3a. Read the file top-to-bottom (≈15 min)

Stop 1 — Imports & config

```
KB_DIR = Path(__file__).parent / "data" / "kb"
PERSIST_DIR = Path(__file__).parent / ".chroma"
COLLECTION_NAME = "cloudnest_kb"
EMBEDDING_MODEL = "sentence-transformers/all-MiniLM-L6-v2"
CHUNK_SIZE = 1000
CHUNK_OVERLAP = 100
```

> "Three constants you will always tune in a real project: where your data lives, where your index lives, and which embedding model. We use `all-MiniLM-L6-v2` because it is small (90 MB), fast (CPU is fine), and good enough for English support docs. For production English-language work, also consider `bge-small-en-v1.5`."

Stop 2 — `load_kb`

```
def load_kb(kb_dir: Path = KB_DIR) -> List[Document]:
    """Load all .md files in the knowledge base directory."""
    if not kb_dir.exists():
        raise FileNotFoundError(f"Knowledge base directory not found: {kb_dir}")
    loader = DirectoryLoader(
        str(kb_dir),
        glob="**/*.md",
        loader_cls=TextLoader,
        loader_kwargs={"encoding": "utf-8"},
    )
    docs = loader.load()
    if not docs:
        raise RuntimeError(f"No markdown files found in {kb_dir}")
    return docs
```

> "LangChain's `DirectoryLoader` returns a list of `Document` objects, each with `.page_content` and `.metadata`. The `metadata['source']` is the filename — we use that for citation later."

Live exercise (≈3 min): have students open one `.md` file in `data/kb/` so they see what raw input looks like.

Stop 3 — Chunking

```
def chunk_documents(docs: List[Document]) -> List[Document]:
    splitter = RecursiveCharacterTextSplitter(
        chunk_size=CHUNK_SIZE,
        chunk_overlap=CHUNK_OVERLAP,
        length_function=len,
    )
    return splitter.split_documents(docs)
```

> "Why chunk? Embeddings have a token limit, and finer-grained chunks mean more relevant retrieval. Why **overlap**? So a sentence that straddles two chunks is not lost. The `Recursive` part means the splitter tries paragraph breaks first, then sentences, then words — it does not blindly cut mid-sentence."
>
> "Rule of thumb: `chunk_size=1000` characters with `overlap=100` is a sane default for English prose. Code or tables need different splitters."

Stop 4 — Embeddings

```
def get_embeddings() -> HuggingFaceEmbeddings:
    return HuggingFaceEmbeddings(
        model_name=EMBEDDING_MODEL,
        model_kwargs={"device": "cpu"},
        encode_kwargs={"normalize_embeddings": True},
    )
```

> "An embedding is a fixed-size vector — for this model, 384 floats — representing the meaning of a piece of text. Texts with similar meaning land near each other in vector space. `normalize_embeddings=True` makes cosine similarity equivalent to dot product, which Chroma uses by default."

Stop 5 — The persistent vectorstore (storytelling moment)

```
def build_vectorstore(chunks, embeddings) -> Chroma:
    return Chroma.from_documents(
        documents=chunks,
        embedding=embeddings,
        persist_directory=str(PERSIST_DIR),
        collection_name=COLLECTION_NAME,
    )

def build_default_vectorstore() -> Chroma:
    embeddings = get_embeddings()
    if PERSIST_DIR.exists() and any(PERSIST_DIR.iterdir()):
        vs = _load_persisted_vectorstore(embeddings)
        try:
            if vs._collection.count() > 0:
                return vs
        except Exception:
            pass
    docs = load_kb()
    chunks = chunk_documents(docs)
    return build_vectorstore(chunks, embeddings)
```

This is your **first storytelling moment**.

> "I want to show you a debugging story I went through with this exact code yesterday. I originally had no `persist_directory` — just `Chroma.from_documents(chunks, embeddings)`. Everything seemed to work. Then Streamlit hot-reloaded and *every retrieval started returning zero documents, silently*. Want to guess why?
>
> *[take 30 seconds of audience guesses]*
>
> Chroma's in-memory mode keeps the collection in a SQLite `memory://` table tied to the Chroma client process. Streamlit's hot-reload re-imported `chromadb`, which created a new client, which had no collection — but `@st.cache_resource` was still handing out the *old* vectorstore object pointing at a dead table. SQLite said `no such table: collections`. The agent honestly told users 'I cannot find anything'. The bug was a Chroma + Streamlit interaction, three layers below where the symptom appeared.
>
> Persisting to disk fixes it: stable file, stable collection, survives reloads, **also** survives `Ctrl+C`."

This story does three things at once: it teaches the gotcha, models how to read a stack trace, and shows that seasoned developers also lose hours to silly things.

3b. Live run (≈10 min)

```
python rag.py
```

Expected output:

```
Loading KB...
  Loaded 5 documents
  Split into 12 chunks
Building vectorstore (downloads model on first run)...

Test query: 'how do I get a refund?'

  [1] 04_refunds_and_cancellation.md
      ...
```

Audience question: "Why did the chunk count (12) come out higher than the document count (5)?" → because chunking subdivides docs.

3c. Quick experiment (≈5 min)

Tell them: "Edit `rag.py` — change the test query to something *not* in the KB, like `how do I bake a sourdough loaf`. Run again. What do you see?"

> "Notice that you still get 3 results — Chroma always returns top-k, **no matter how irrelevant**. This is a critical lesson: the retriever has no notion of 'relevant or not'. It always answers. It is the LLM's job to decide if the chunks are useful. We will see how the agent does that next."

Break (1:05 – 1:15)

Have a slide up with the next block's preview. Do not try to debug individual setups during the break — point students at a shared troubleshooting doc.

Block 4 — `agent.py` deep dive (1:15 – 2:00)

Goal: students understand the ReAct loop, tool schemas, and the system prompt.

4a. Concept (≈5 min)

Whiteboard the ReAct loop:

```
think -> decide tool -> call tool -> observe result -> think -> ... -> answer
```

> "ReAct = **Reasoning** + **Acting**. The model alternates between two modes. LangGraph implements this as a state machine. We use the prebuilt `create_react_agent` because writing it from scratch is a 200-line graph definition we do not have time for — but the source is readable, look at it after the workshop."

4b. File walkthrough (≈25 min)

Stop 1 — The system prompt

```
SYSTEM_PROMPT = """You are CloudNest Support, the official virtual assistant for \
CloudNest — a cloud file-storage platform.

# Role
Help customers resolve questions about their CloudNest account, subscription, and
product usage. Be helpful, accurate, and professional at all times. Never
speculate or fabricate policy details.

# Tool usage
1. search_knowledge_base — call this BEFORE answering any product question.
   - Treat returned chunks as authoritative.
   - Call this tool at most twice per user turn.
2. create_support_ticket — call when the user asks for a human or hits an issue
   the KB cannot resolve.
3. No tool — for greetings/thanks, reply directly.

# Style
- Be concise: 2-4 sentences typically.
- Cite the source filename when you used search_knowledge_base.
"""
```

This is your **second storytelling moment**.

> "This prompt did not start this clean. Yesterday I had three lines: 'Be helpful. Use search. Be concise.' The agent then called `search_knowledge_base` four times in a row, each time with a slightly different query, and concluded the KB had nothing — even though the answer was sitting right there. Why? Because small models, faced with a tool result they do not fully trust, retry instead of read.
>
> The fix is in this prompt: notice the line 'Treat returned chunks as authoritative' and 'at most twice per user turn'. Prompts are not just personality — they are guardrails. Each constraint here came from an observed failure."

Stop 2 — The tools

```
@tool
def search_knowledge_base(query: str) -> str:
    """Search CloudNest's help center for information about pricing, features,
    troubleshooting, account management, sharing, refunds, and policies.
    Use this for any question about how CloudNest works.
```

```
    Args:
        query: A natural-language search query.
    """
    if _RETRIEVER is None:
        return "Retriever not initialized."
    docs = _RETRIEVER.invoke(query)
    if not docs:
        return "No relevant information found in the knowledge base."
    formatted = []
    for i, d in enumerate(docs, 1):
        source = d.metadata.get("source", "unknown").replace("\\", "/").split("/")[-1]
        formatted.append(f"[Source {i}: {source}]\n{d.page_content}")
    return "\n\n---\n\n".join(formatted)
```

Three things to highlight:

1. **The `@tool` decorator turns a Python function into a tool the LLM can call.**
2. **The docstring is the tool's description** — the LLM reads it to decide *when* to use the tool. **Bad docstring = bad tool selection.**
3. **The tool returns a string.** Even though it is structured (numbered sources), it is a single string. The LLM reads it like a document.

Audience question: "Why does the docstring list 'pricing, features, troubleshooting, account management, sharing, refunds, and policies'?" → Because the LLM uses these words to match user intent to this tool. Generic docstrings ("searches the database") cause the LLM to either over-call or never-call.

Stop 3 — The `_RETRIEVER` global hack

```
# Module-level retriever holder. build_agent() sets this before tools run.
_RETRIEVER = None
```

> "This is ugly and I want to be honest with you about why it is like this. The `@tool` decorator captures the function at *import time*. If we used a closure — defining `search_knowledge_base` inside `build_agent` so it could capture `retriever` from the enclosing scope — the decorator's docstring extraction breaks across LangChain versions. We hit that exact `ValueError: Function must have a docstring if description not provided` error yesterday. Module-level tools with a global retriever holder is the path of least resistance for this version. It is a real-world example of *the cleanest design lost to a library constraint*."

Stop 4 — `build_agent`

```
def build_agent(retriever, model_name="llama-3.1-8b-instant", temperature=0.2):
    global _RETRIEVER
    _RETRIEVER = retriever
    llm = ChatGroq(model=model_name, temperature=temperature)
    tools = [search_knowledge_base, create_support_ticket]
    memory = MemorySaver()
```

```
    agent = create_react_agent(
        model=llm,
        tools=tools,
        state_modifier=SYSTEM_PROMPT,
        checkpointer=memory,
    )
    return agent
```

Your **third storytelling moment** — the version trap.

> "See `state_modifier`? In `langgraph >= 0.3` this argument was renamed to `prompt`. We are pinned to `0.2.50`, so we use `state_modifier`. If you copy this code into a project with a newer langgraph, you get `TypeError: got an unexpected keyword argument 'state_modifier'`. **Lesson: always run `python -c 'import inspect; from langgraph.prebuilt import create_react_agent; print(inspect.signature(create_react_agent))'` before trusting a tutorial.**"

Stop 5 — The CLI test

> "Notice the `if __name__ == '__main__':` block. This lets you test the agent **without Streamlit** — pure Python, three questions, watch the trace. Always have a CLI entry point alongside your UI. It is how you debug at 2 AM when Streamlit is misbehaving."

4c. Live run (≈10 min)

Have students set their key in `.env`:

```
GROQ_API_KEY=gsk_their_key_here
```

Then run:

```
python agent.py
```

Expected output:

```
Building vectorstore...
Building agent...

User: Hi!
Bot: Hello! How can I help you with CloudNest today?

User: What's the difference between the Pro and Business plans?
Bot: Pro is $9.99/month with 2 TB storage and 24-hour support;
     Business starts at $15/user/month...

User: I want to talk to a human, my files are corrupted and I can't access them.
Bot: I have created ticket CN-...
```

Audience exercise (5 min): "Add a fourth question to the questions list. Try one that is clearly out of scope — 'What is the weather in Cairo?' — and see how the agent handles it."

4d. Visualize the graph (≈5 min, optional but loved)

In a Python REPL:

```
from rag import build_default_vectorstore
from agent import build_agent
agent = build_agent(build_default_vectorstore().as_retriever())
print(agent.get_graph().draw_ascii())
```

Students see the actual state machine. This makes "the agent is a graph" concrete.

Block 5 — `app.py` deep dive (2:00 – 2:35)

Goal: students understand `st.cache_resource`, `st.session_state`, `st.chat_input`, and streaming.

5a. Concept (≈3 min)

> "Streamlit's mental model: **every interaction reruns the entire script top-to-bottom**. That is why we need caching — we do not want to re-embed 12 chunks on every keystroke. And it is why we need session state — variables do not survive between reruns unless they are in `st.session_state`."

5b. File walkthrough (≈20 min)

Stop 1 — Defaults

```
MODEL_NAME = "llama-3.1-8b-instant"
TEMPERATURE = 0.2
TOP_K = 3
```

Your **fourth storytelling moment**.

> "I tried switching this to `llama-3.3-70b-versatile` yesterday — bigger model, surely better. The agent immediately threw `BadRequestError: tool_use_failed`. Groq's API rejected the model's output because Llama 3.3 emits `<function=...></function>` XML instead of structured JSON. **Bigger does not mean better for tool-calling.** Tool-calling is a *protocol* — the model has to follow the API spec exactly. Always test tool-calling with the actual model before committing to it."

Stop 2 — Secrets

```
if "GROQ_API_KEY" in st.secrets:
    os.environ["GROQ_API_KEY"] = st.secrets["GROQ_API_KEY"]
```

```
if not os.environ.get("GROQ_API_KEY"):
    st.error("GROQ_API_KEY is not set...")
    st.stop()
```

> "Two patterns worth memorizing: `st.secrets` is auto-loaded from `.streamlit/secrets.toml` locally and from the Cloud secrets UI in deployment. And `st.stop()` is how you halt rendering when a precondition fails."

Stop 3 — Caching

```
@st.cache_resource(show_spinner="Loading knowledge base and embedding model...")
def get_vectorstore():
    return build_default_vectorstore()

@st.cache_resource(show_spinner="Initializing agent...")
def get_agent(_vectorstore, model_name, temperature, k):
    retriever = _vectorstore.as_retriever(search_kwargs={"k": k})
    return build_agent(retriever, model_name=model_name, temperature=temperature)
```

> "`@st.cache_resource` is for **objects you want exactly one of**, like a vectorstore or model. `@st.cache_data` is for **data you want copies of**, like dataframes. Use the right one — they have different invalidation rules."
>
> "The leading underscore on `_vectorstore` tells Streamlit *do not hash this argument when computing the cache key*. Without it, you get `UnhashableParamError` because Chroma objects are not hashable. The other args — `model_name`, `temperature`, `k` — *are* hashed, so changing any of them rebuilds the agent."

Stop 4 — Session state

```
if "messages" not in st.session_state:
    st.session_state.messages = []
if "thread_id" not in st.session_state:
    import uuid
    st.session_state.thread_id = str(uuid.uuid4())
```

> "Two pieces of per-user state: the chat history (for rendering) and the LangGraph thread ID (for the agent's memory). Each browser tab gets its own UUID, so two students using the same deployed app do not see each other's conversations."

Stop 5 — Streaming

```
for event in agent.stream(
    {"messages": [("user", prompt)]},
    config=config,
    stream_mode="values",
```

```
):
    last_msg = event["messages"][-1]
    if hasattr(last_msg, "tool_calls") and last_msg.tool_calls:
        for tc in last_msg.tool_calls:
            tool_calls_seen.append({"name": tc["name"], "args": tc["args"]})
            placeholder.info(f"Calling `{tc['name']}`...")
    if last_msg.type == "ai" and last_msg.content:
        final_text = last_msg.content
```

> "This is the difference between *boring* and *magical*. `agent.invoke(...)` would block for 8 seconds with nothing on screen. `agent.stream(...)` yields each step of the graph as it happens, and we update the UI as tool calls fire. **For LLM apps, streaming is not a luxury — it is table stakes.**"

5c. Live run (≈10 min)

```
streamlit run app.py
```

Browser opens. Students try:

1. "tell me about pricing" — expect plan list with citation.
2. "and the cheapest one?" — expect "Free" — proves memory works.
3. "I want to speak to a human, my files are corrupted" — expect ticket creation.

The classic mistake: someone will type `python app.py` instead of `streamlit run app.py`. Have your slide ready: *"Streamlit apps must be launched with `streamlit run`. The `python` command produces a wall of `missing ScriptRunContext` warnings — that is the symptom."*

5d. The Ctrl+C reset story (≈2 min)

> "If you change the system prompt in `agent.py`, **Streamlit will not pick it up** — `@st.cache_resource` only invalidates when its arguments change. You have to stop the server with `Ctrl+C` and restart. That is a real production gotcha when promoting prompt changes."

Block 6 — Deploy to Streamlit Community Cloud (2:35 – 2:50)

Goal: every student leaves with a public URL.

Steps (do them live, students follow along)

1. **Push to GitHub** (5 min). Create a new public repo, push the project. Critical: **do not commit `.streamlit/secrets.toml` or `.env`**. Verify with `git status` that your `.gitignore` is working.
2. **Connect Streamlit Cloud to your GitHub** (3 min). Go to https://share.streamlit.io, click "New app", pick the repo, branch `main`, main file `app.py`.

3. **Add secrets** (2 min). In the app settings, paste:

```
GROQ_API_KEY = "gsk_..."
```

4. **Deploy** (5 min). Wait for the build. First boot takes ~2 minutes (pip install + first KB embed). When it works, share your link in the workshop chat.

Talking points while it builds

- "First cold start is slow because: pip install (~30 s), HuggingFace model download (~10 s), KB embedding (~30 s). After that, every visit is fast."
- "The free tier gives ~1 GB RAM. Watch the Cloud logs — `torch` and `sentence-transformers` are the heavy components."
- "Apps sleep after ~7 days of no traffic. First visitor after a sleep eats the cold start again. For a demo this is fine."

Block 7 — Q&A and extensions (2:50 – 3:00)

End with three concrete things students can do tomorrow.

Three extensions worth attempting

1. **Add a third tool**, e.g. `check_order_status(order_id: str)` returning mock JSON. Shows how to add tools without rewriting the agent.
2. **Swap the embedding model** to `bge-small-en-v1.5` — same API, better quality. Shows how interchangeable the layer is.
3. **Add a real evaluation set**: 10 KB questions + expected sources, score retrieval recall@3. Moves them from "vibes" to "metrics".

A parting question for the audience

> "We did three things in three hours: built a retriever, wrapped it with an agent, and wrapped *that* with a UI. Which of the three would you say is the hardest to make production-ready, and why?"
>
> *(Right answer: the retriever — chunking, evaluation, refresh, multilingual… the LLM and UI are commoditized; the data layer is where the real engineering lives.)*

Common student questions — pre-prepared answers

Question	Answer

"Can I use OpenAI instead of Groq?"	Yes — `from langchain_openai import ChatOpenAI; llm = ChatOpenAI(model="gpt-4o-mini")`. Set `OPENAI_API_KEY` instead of `GROQ_API_KEY`. Agent code does not change.
"Where does the chat history actually live?"	Two places. Streamlit's `st.session_state.messages` for rendering. LangGraph's `MemorySaver` keyed by `thread_id` for the agent's reasoning context.
"Is `MemorySaver` durable?"	No. In-memory only, lost on restart. Swap for `SqliteSaver` for persistence.
"Can I add PDFs?"	Yes — `PyPDFLoader` is already in `requirements.txt`. The `load_pdf` function in `rag.py` works.
"How do I evaluate this?"	Build a CSV of `(question, expected_source_file)` pairs. For each, run the retriever and check if the expected file appears in the top-k results. That is *retrieval recall@k*.
"Why ReAct and not function-calling directly?"	Same thing under the hood for modern LLMs. ReAct just provides a conventional state machine wrapper that handles loops, max-iterations, and tool dispatch.
"What is the cheapest way to deploy this?"	Streamlit Community Cloud (free) → Hugging Face Spaces (free) → Render/Fly.io ($5–7/mo) → AWS App Runner. In that order of effort.

Contingency plans

Things that will break, and how to recover without losing the room.

"My `.chroma/` directory is corrupted"

PowerShell:

```
Remove-Item -Recurse -Force .chroma
```

Bash/zsh:

```
rm -rf .chroma
```

Then re-run the script.

"I get `tool_use_failed` from Groq"

You are on `llama-3.3-70b-versatile`. Switch `MODEL_NAME` back to `llama-3.1-8b-instant` in `app.py`.

"I get `state_modifier` errors"

Your langgraph is newer than 0.2.x — change `state_modifier=SYSTEM_PROMPT` to `prompt=SYSTEM_PROMPT` in `agent.py`.

"I get `Function must have a docstring if description not provided`"

Your tool function lost its docstring. Restore the triple-quoted docstring — LangChain reads it as the tool description.

"The 30s embedding download times out on conference Wi-Fi"

Pre-stage the cache: before the workshop, copy your `~/.cache/huggingface/hub/models--sentence-transformers--all-MiniLM-L6-v2/` to a USB stick. Students drop it into the same path and skip the download. Or do the prereq step from the email seriously.

"Streamlit Cloud build is failing"

99% of the time it is `chromadb` failing to install on Python 3.13. Add a `runtime.txt` containing `python-3.11`.

"Two students share the same Groq key and they are getting rate-limited"

Free Groq accounts give individual rate limits — every student needs their *own* key. Five minutes of awkward signups beats an hour of mystery 429s.

Slides cheat sheet

If you are making a deck, you only need these:

1. Title + your name
2. Demo screenshot
3. The whiteboard pipeline diagram (RAG)
4. The three-file architecture
5. ReAct loop diagram
6. `rag.py` outline (5 numbered functions)
7. `agent.py` outline (system prompt → tools → build_agent)
8. `app.py` outline (cache → session → render → stream)
9. Deploy checklist
10. Three extensions

Ten slides + live coding for 3 hours. Resist the urge to make more.

Final advice for the instructor

- **The bug stories are the workshop's emotional anchors.** Without them, this is a tutorial. With them, it is a craft demonstration. Tell them. Slow down for them.
- **The minute someone's screen has a red error, pull a "field trip" — read the error out loud as a**

class. Every error read together is worth ten read alone.
- **End on time.** A workshop that runs over loses the deploy section, which is the part students will *brag about* later. Trim demo time, not deploy time.
- **Have your finished, deployed app open on a backup device.** If your laptop dies, you can still demo from your phone.

The codebase is genuinely well-suited for this — concise enough to read in real time, real enough to teach lessons that stick.

Good luck.

— *Eng. Youssef Bastawisy*